

Universidade Federal do Pará
Faculdade de Sistemas de Informação

FasiTech

Fundamentos Científicos do Sistema

Módulo de Recuperação com Geração Aumentada (RAG)

Módulo de Extração Inteligente de Documentos (ACC)

Sumário

1	Introdução	3
2	Módulo RAG — Diretor Virtual	4
2.1	Fundamentos: Retrieval-Augmented Generation	4
2.1.1	Recuperação Densa com Bi-Encoders	4
2.1.2	Busca Híbrida: Semântica + BM25	5
2.2	Estratégias de Chunking	5
2.2.1	Chunking Básico com Sobreposição	5
2.2.2	Chunking Contextual de Seção (Melhoria 1)	5
2.2.3	Chunking Pai-Filho (<i>Small-to-Big</i>)	7
2.3	Cache Semântico	8
2.3.1	Motivação e Arquitetura	8
2.3.2	Ciclo de Vida das Entradas	8
2.4	Filtragem de Domínio	8
2.5	Detecção de Ambiguidade Temporal (Melhoria 2)	9
2.6	Expansão de Consulta por Siglas	11
2.7	Visão Geral do Pipeline RAG	12
3	Módulo ACC — Extração Inteligente de Documentos	13
3.1	Contexto e Desafio	13
3.2	Fundamentos: Document AI e Multimodal LLMs	13
3.2.1	Modelos Multimodais	13
3.2.2	Processamento de PDFs com pdf2image e Poppler	14
3.3	Arquitetura do Módulo ACC	15
3.4	Prompt Engineering para Extração Estruturada	15
3.4.1	Extração do Total por Expressão Regular	16
3.5	Análise de Confiabilidade e Limitações	16
3.5.1	Fontes de Erro	16
3.5.2	Escalabilidade	16

4	Infraestrutura e Integração	17
4.1	Banco de Dados Vetorial: LanceDB	17
4.2	Modelos de Linguagem e Embedder	17
4.3	Padrão de Projeto: Singleton com Inicialização Lazy	18
5	Trabalhos Futuros	19

Capítulo 1

Introdução

O sistema FasiTech é uma plataforma web institucional desenvolvida para centralizar fluxos acadêmicos da Faculdade de Sistemas de Informação (FASI) da UFPA. Entre seus módulos, dois se apoiam diretamente em técnicas de Inteligência Artificial:

1. **Diretor Virtual** — assistente conversacional baseado em *Retrieval-Augmented Generation* (RAG) que responde perguntas sobre regulamentos, cronogramas e documentos acadêmicos.
2. **Módulo ACC** — extrator automático de cargas horárias de certificados enviados em formato PDF, usando Modelos de Linguagem Multimodais (*Multimodal Large Language Models*).

Este documento registra a ciência aplicada em cada componente, servindo de referência para manutenção, evolução e publicação futura dos resultados.

Capítulo 2

Módulo RAG — Diretor Virtual

2.1 Fundamentos: Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) é uma arquitetura que combina recuperação de documentos com geração de linguagem natural. A formulação original foi proposta por Lewis et al. (2020): dado um modelo generativo $p_\theta(y \mid x)$ e um índice de documentos $\mathcal{D}$, o modelo recupera os documentos mais relevantes $\mathbf{z} = \text{Top-}k(q, \mathcal{D})$ e condiciona a geração sobre eles:

$$p(y \mid x) = \sum_{z \in \mathbf{z}} p_\eta(z \mid x) p_\theta(y \mid x, z) \quad (2.1)$$

onde $p_\eta(z \mid x)$ é a probabilidade do documento z ser relevante para a consulta x , calculada via busca densa, e p_θ é o LLM gerador.

Por que RAG e não fine-tuning?

Fine-tuning parametriza o conhecimento no modelo, tornando atualizações custosas e sem rastreabilidade de fonte. RAG externaliza o conhecimento nos documentos, permitindo atualização imediata sem retreinamento e garantindo que respostas sejam fundamentadas em trechos verificáveis.

2.1.1 Recuperação Densa com Bi-Encoders

A recuperação densa (DR — *Dense Retrieval*) representa consultas e documentos como vetores densos em um espaço semântico compartilhado. O trabalho seminal de Karpukhin et al. (2020) (DPR — *Dense Passage Retrieval*) demonstrou que bi-encoders treinados por aprendizado contrastivo superam a busca esparsa BM25 em domínios factuais.

A similaridade entre a consulta q e um documento d é calculada por:

$$\text{sim}(q, d) = \cos(E_Q(q), E_D(d)) = \frac{E_Q(q) \cdot E_D(d)}{\|E_Q(q)\| \|E_D(d)\|} \quad (2.2)$$

No sistema FasiTech, o embedder é o **Ollama Embedder** (`nomic-embed-text` (NUS-SBAUM et al., 2024)), executado localmente via servidor Ollama (OLLAMA, 2023) (`OLLAMA_HOST`, padrão `localhost:11434`), produzindo vetores de 768 dimensões. Todos os vetores são normalizados pelo serviço (norma $L2 = 1$) antes de serem armazenados no LanceDB, permitindo que a similaridade cosseno seja computada como produto escalar simples — $\text{sim}(q, d) = E(q) \cdot E(d)$.

2.1.2 Busca Híbrida: Semântica + BM25

Busca híbrida combina recuperação densa (semântica) com recuperação esparsa baseada em frequência de termos (ROBERTSON et al., 1994). O score BM25 para um documento d dado uma consulta q com termos $\{q_i\}$ é:

$$\text{BM25}(d, q) = \sum_i \text{IDF}(q_i) \cdot \frac{f(q_i, d) (k_1 + 1)}{f(q_i, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)} \quad (2.3)$$

onde k_1 e b são hiperparâmetros de saturação de frequência e normalização por comprimento do documento, respectivamente.

O banco vetorial **LanceDB** utilizado no FasiTech configura `SearchType.hybrid`, combinando os dois rankings via *Reciprocal Rank Fusion* (RRF) (CORMACK; CLARKE; BUETTCHER, 2009):

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + r(d)} \quad (2.4)$$

onde $r(d)$ é a posição do documento no ranking r e $k = 60$ é constante. Além disso, o módulo `_keyword_search` realiza busca adicional por palavras-chave como complemento pós-semântico.

2.2 Estratégias de Chunking

2.2.1 Chunking Básico com Sobreposição

A fragmentação de documentos (*chunking*) é etapa crítica no pipeline RAG. O método original no FasiTech divide textos em blocos de 800 caracteres com sobreposição (*overlap*) de 150 caracteres, respeitando quebras de parágrafo. Esta abordagem, denominada *fixed-size chunking with overlap*, minimiza a perda de contexto nas fronteiras dos chunks.

2.2.2 Chunking Contextual de Seção (Melhoria 1)

A limitação do chunking simples é a perda do contexto hierárquico do documento. Quando um chunk pertence à seção “Período Letivo 2026.2”, o título dessa seção pode estar

em um chunk anterior, tornando a recuperação dependente de que o usuário mencione explicitamente “2026.2” na consulta.

Inspirado no *Contextual Retrieval* proposto pela Anthropic (ANTHROPIC, 2024) e no trabalho de Shi et al. (2023) sobre prepend de contexto para melhoria de recuperação, o FasiTech implementa o método `_split_with_section_context`:

1. O documento Markdown é percorrido linha a linha.
2. Cabeçalhos (`#`, `##`, `###`, `####`) são detectados via expressão regular `^(#{1,4})\s+(.+)` e armazenados como `section_header` corrente.
3. Cada chunk gerado pelo método base (`_split_into_chunks`) recebe um *prefixo contextual*:

[Documento: X | Seção: Y]\n\n<chunk>

4. O par (`section_header`, `chunk_contextualizado`) é retornado; o campo `section` é persistido nos metadados do LanceDB.
5. O chunk contextualizado — não o texto bruto — é embeddado e indexado.

```

1 lines = text.split("\n")
2 heading_re = re.compile(r"^(#{1,4})\s+(.+)")
3
4 # Agrupa linhas por secao
5 sections: List[tuple[str, List[str]]] = []
6 current_header, current_lines = "", []
7 for line in lines:
8     m = heading_re.match(line)
9     if m:
10         if current_lines:
11             sections.append((current_header, current_lines))
12             current_header = m.group(2).strip()
13             current_lines = []
14         else:
15             current_lines.append(line)
16
17 # Gera chunks contextualizados
18 result: List[tuple[str, str]] = []
19 for sec_header, sec_lines in sections:
20     block = "\n".join(sec_lines).strip()
21     raw_chunks = ChatbotService._split_into_chunks(block,
22                                                     chunk_size, overlap)

```

```

22     for chunk in raw_chunks:
23         prefix = f"[Documento: {doc_name} | Secao: {sec_header}]\n\n"
24         contextualized = (prefix + chunk) if sec_header else
            chunk
25         result.append((sec_header, contextualized))

```

Listing 2.1: Núcleo de `_split_with_section_context` (`rag_ppc.py`).

O efeito é que o embedding do chunk carrega informação da seção pai, aumentando a similaridade com consultas que referenciam aquela seção, mesmo que o chunk em si não a mencione.

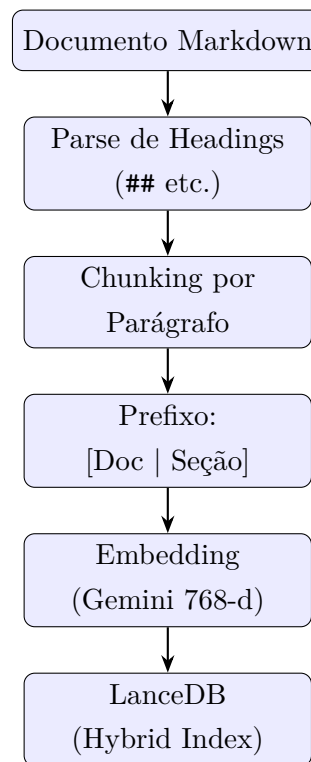


Figura 2.1: Pipeline de indexação com chunking contextual de seção.

2.2.3 Chunking Pai-Filho (*Small-to-Big*)

Como direção futura, o FasiTech pode adotar o *Parent-Child Chunking* introduzido no LlamaIndex (LLAMAINDEX, 2023): chunks pequenos (200 chars) são indexados para busca precisa, mas o contexto entregue ao LLM é o chunk pai (1 200 chars) que contém o trecho recuperado. Isso reduz o ruído na busca vetorial enquanto mantém contexto suficiente para a geração.

2.3 Cache Semântico

2.3.1 Motivação e Arquitetura

O cache semântico armazena pares (pergunta, resposta) avaliados positivamente e os serve em consultas futuras semanticamente similares, eliminando chamadas ao LLM para perguntas recorrentes. Esta abordagem é referenciada em Bang et al. (2023) e Zhu et al. (2023).

O sistema FasiTech implementa um cache em duas camadas no LanceDB:

Tabela 2.1: Parâmetros do cache semântico do FasiTech.

Parâmetro	Descrição	Valor
Threshold de similaridade	Cosseno mínimo para hit de cache	0,90
Avaliação mínima (<i>trusted</i>)	Média de notas para promover entrada	4,4/5
Avaliações para promoção	Qtd. mínima de feedbacks	2
TTL <i>trusted</i>	Validade de entrada confiável	30 dias
TTL <i>candidate</i>	Validade de entrada candidata	14 dias

2.3.2 Ciclo de Vida das Entradas

Uma entrada percorre o seguinte ciclo:

1. **Candidate** — criada após primeira avaliação positiva (nota ≥ 3).
2. **Trusted** — promovida quando $\bar{r} \geq 4,4$ com $n \geq 2$ avaliações.
3. **Servida** — retornada diretamente ao usuário sem chamar o LLM.
4. **Expirada** — removida após TTL ou se nota ≤ 2 na última avaliação.

A função de confiança C combina qualidade e volume:

$$C = \frac{\bar{r} - 1}{4} \cdot \min\left(1, \frac{n}{n_{\min}}\right) \quad (2.5)$$

Adicionalmente, cada entrada armazena o `documents_hash`, garantindo que mudanças nos documentos fontes invalidem automaticamente o cache.

2.4 Filtragem de Domínio

Antes de acionar o RAG, o sistema verifica se a pergunta pertence ao domínio acadêmico para evitar respostas sobre tópicos fora de escopo.

A estratégia é bifásica: (1) detecção de palavras-chave acadêmicas (ex.: “TCC”, “matrícula”, “ACC”) ajusta o limiar de similaridade para mais permissivo ($\tau = 0,50$); (2) na ausência de palavras-chave, um limiar mais estrito é aplicado ($\tau = 0,72$) para evitar que perguntas genéricas em português sejam processadas por similaridade superficial de idioma.

O score combinado usa 40% da melhor similaridade e 60% da média dos três primeiros resultados:

$$S = 0,4 \cdot \text{sim}_{\max} + 0,6 \cdot \bar{\text{sim}}_3 \quad (2.6)$$

2.5 Detecção de Ambiguidade Temporal (Melhoria 2)

Estudos em busca conversacional demonstram que perguntas ambíguas — aquelas que carecem de entidades contextuais necessárias para uma resposta precisa — devem ser respondidas com uma pergunta de esclarecimento antes da recuperação (ALIANNEJADI et al., 2021).

No contexto do FasiTech, a ambiguidade temporal ocorre quando o usuário pergunta sobre prazos sem especificar o período letivo. O método `_needs_temporal_clarification` implementa a seguinte lógica:

$$\text{clarify}(q) = \underbrace{\exists t \in T_{\text{trigger}} : t \in q}_{\text{gatilho temporal}} \wedge \underbrace{\nexists e \in E_{\text{period}} : e \in q}_{\text{ausência de entidade}} \quad (2.7)$$

Os conjuntos são implementados como `frozenset` para busca $O(1)$:

```

1 _TEMPORAL_TRIGGERS: frozenset = frozenset({
2     "periodo", "prazo", "submissao", "entrega", "data", "quando",
3     "limite", "cronograma", "calendario", "vencimento", "ate
4     quando",
5     "deadline", "datas", "prazos", "agenda",
6 })
7 _TEMPORAL_ENTITIES: frozenset = frozenset({
8     # Identificadores de periodo letivo: 2024.1 ... 2028.4
9     "2024.1", "2024.2", "2024.3", "2024.4",
10    "2025.1", "2025.2", "2025.3", "2025.4",
11    "2026.1", "2026.2", "2026.3", "2026.4",
12    "2027.1", ..., "2028.4",
13    # Expressoes naturais de periodo
14    "primeiro periodo", "segundo periodo", "semestre atual",
15    "proximo semestre", "este semestre",
16 })

```

```

16
17 @staticmethod
18 def _needs_temporal_clarification(question: str) -> bool:
19     q_lower = question.lower()
20     has_trigger = any(t in q_lower for t in _TEMPORAL_TRIGGERS)
21     has_entity = any(e in q_lower for e in _TEMPORAL_ENTITIES)
22     return has_trigger and not has_entity

```

Listing 2.2: Constantes de detecção temporal (rag_ppc.py).

Quando $\text{clarify}(q) = \top$, o método `_get_available_periods` varre todos os chunks do LanceDB em busca do padrão `\b(20\d{2}\.[1-4])\b` e retorna os períodos reais indexados:

```

1 def _get_available_periods(self) -> List[str]:
2     period_re = re.compile(r"\b(20\d{2}\.[1-4])\b")
3     found: set = set()
4     df = lancedb.connect(self.db_url).open_table("recipes").
        to_pandas()
5     for row in df.itertuples():
6         p = json.loads(row.payload)
7         for m in period_re.findall(p.get("content", "")):
8             found.add(m)
9     return sorted(found)    # ex.: ['2026.2', '2026.3', '2026.4']

```

Listing 2.3: Descoberta dinâmica de períodos (rag_ppc.py).

A resposta de esclarecimento é formatada em Markdown (renderizado no frontend via `react-markdown + remark-gfm`):

```

1 period_list = ", ".join(available_periods)    # ex.: 2026.2,
        2026.3, 2026.4
2 clarification = (
3     f"Para qual **período letivo** voce deseja as informacoes?\n\n"
4     f"Períodos disponiveis nos documentos: **{period_list}**\n\n"
5     f"Por favor, especifique o período (ex.: *2026.2*) para que"
6     f"eu "
7     f"possa te dar uma resposta precisa."
8 )

```

Listing 2.4: Mensagem de esclarecimento gerada dinamicamente.

Esta abordagem elimina chamadas desnecessárias ao LLM antes de executar o pipeline RAG completo com contexto insuficiente.

Ganho esperado

Esta abordagem elimina a necessidade de o usuário reformular a pergunta. Em vez de receber uma resposta incompleta ou mesclada entre períodos, o sistema pede esclarecimento na primeira interação, reduzindo o número médio de turnos para resposta de ≈ 2 para 1.

2.6 Expansão de Consulta por Siglas

O método `_expand_query` implementa um mapa de siglas para seus expandidos semânticos (ex.: “ACC” $\rightarrow$ “Atividade Curricular Complementar atividades complementares ACG extracurricular”).

Esta técnica é variante do *Query Expansion* estudado em Carpineto e Romano (2012): ao adicionar termos sinônimos à consulta antes do embedding, aumenta-se o recall da busca vetorial sem modificar o banco de vetores.

2.7 Visão Geral do Pipeline RAG

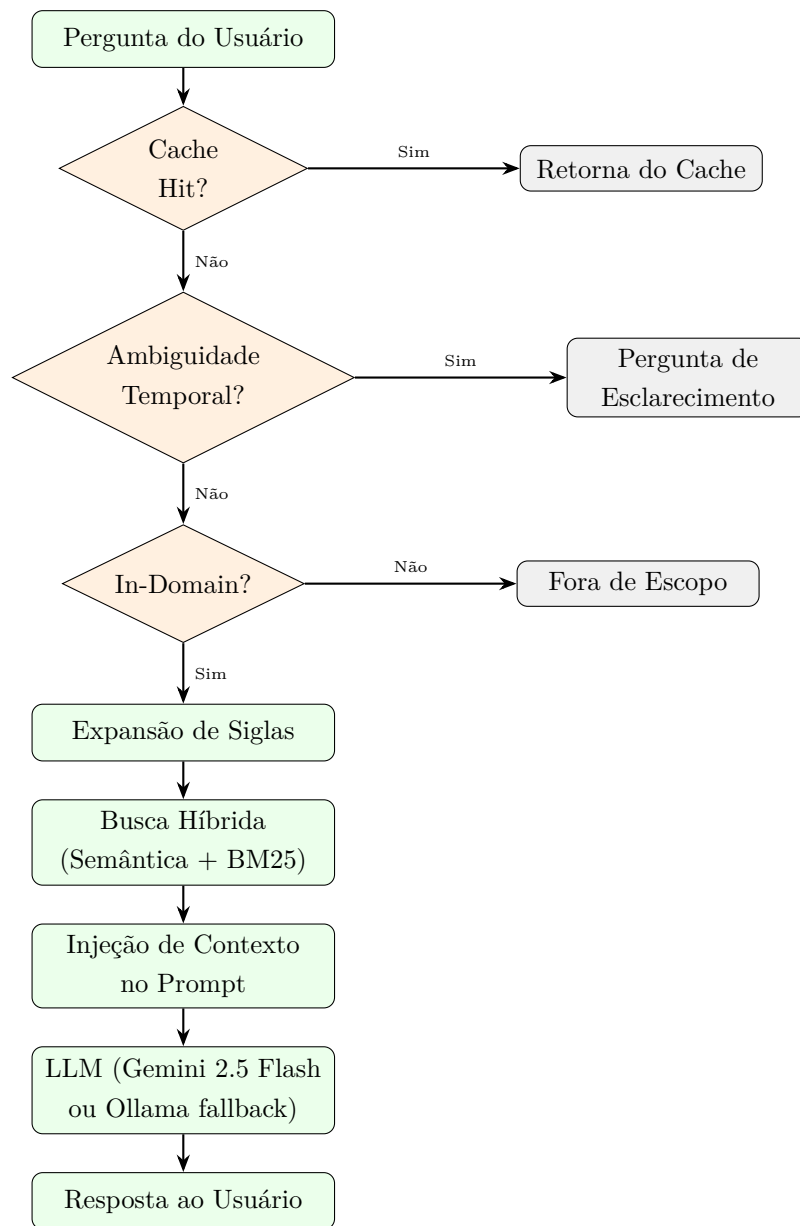


Figura 2.2: Fluxo completo do pipeline RAG do Diretor Virtual.

Capítulo 3

Módulo ACC — Extração Inteligente de Documentos

3.1 Contexto e Desafio

As Atividades Complementares de Graduação (ACC/ACG) são requisito curricular para integralização do curso de Sistemas de Informação. Historicamente, a validação dessas atividades é feita manualmente por servidores, que examinam cada certificado entregue pelo aluno e somam as cargas horárias.

O processo é:

- **Heterogêneo** — certificados variam em formato, layout, idioma e fonte.
- **Propenso a erros** — leitura manual de dezenas de certificados por aluno.
- **Não escalável** — custo cresce linearmente com o número de alunos.

O módulo ACC do FasiTech automatiza a extração da carga horária via *Document AI* com Modelos de Linguagem Multimodais.

3.2 Fundamentos: Document AI e Multimodal LLMs

Document AI é o campo de estudo da compreensão automática de documentos não-estruturados (CUI et al., 2021). Diferentemente de NLP clássico — que opera sobre texto puro —, Document AI lida com a informação visual e estrutural de documentos (layout, tabelas, fontes, posição do texto).

3.2.1 Modelos Multimodais

Modelos de linguagem multimodais (*Multimodal LLMs*) processam imagens e texto de forma conjunta, sem necessidade de OCR separado. O **Gemini** (GEMINI TEAM et al.,

2023) é um modelo multimodal nativo que interpreta texto, imagens e PDFs em uma única passagem, sendo especialmente adequado para certificados escaneados.

A geração de resposta y dado um documento D (página PDF) e uma instrução I pode ser expressa como:

$$y = \arg \max_y p_\theta(y \mid D, I) \quad (3.1)$$

onde D é representado como sequência de tokens visuais extraídos por um *Vision Encoder* e I é o prompt de instrução.

3.2.2 Processamento de PDFs com pdf2image e Poppler

Quando o modelo não suporta PDF nativo, a conversão para imagens de alta resolução (300 DPI) via *pdf2image/Poppler* é prática padrão (BLECHER et al., 2023). O FasiTech oferece suporte a ambos os modos:

1. **PDF direto** — enviado como objeto **File** ao Gemini, que processa nativamente todas as páginas.
2. **PDF → imagem** — cada página é convertida a PNG 300 DPI e enviada individualmente ao modelo via **Image**.

3.3 Arquitetura do Módulo ACC

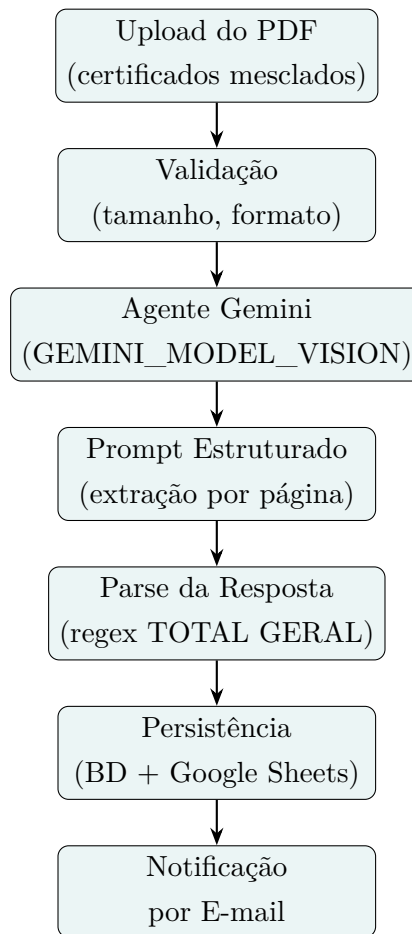


Figura 3.1: Pipeline de processamento do Módulo ACC.

3.4 Prompt Engineering para Extração Estruturada

O desempenho de LLMs em tarefas de extração de informação depende criticamente da qualidade do prompt (BROWN et al., 2020). O FasiTech utiliza um prompt de *few-shot* implícito com especificação de:

- **Papel e tarefa** — “Você é um especialista em extrair informações de certificados acadêmicos”.
- **Formatos esperados** — lista de padrões comuns (“40h”, “40 horas”, “CH: 40h”, etc.).
- **Formato de saída** — estrutura canônica por página e linha TOTAL GERAL: X horas.
- **Tratamento de casos-limite** — carga horária em dias (8h/dia), múltiplas cargas na página, ausência de carga.

3.4.1 Extração do Total por Expressão Regular

Após a resposta do modelo, o sistema executa parsing determinístico para extrair o total numérico, mitigando alucinações:

```

1 padroes = [
2     r'TOTAL GERAL:\s*(\d+(?:,\d+)?)\s*horas?',
3     r'Total Geral:\s*(\d+(?:,\d+)?)\s*horas?',
4     r'TOTAL:\s*(\d+(?:,\d+)?)\s*horas?',
5 ]
6 for padrao in padroes:
7     match = re.search(padrao, texto, re.IGNORECASE)
8     if match:
9         horas = match.group(1).replace(',', '.', '')
10        return f"TOTAL GERAL: {horas} horas"

```

Listing 3.1: Extração de total com regex.

3.5 Análise de Confiabilidade e Limitações

3.5.1 Fontes de Erro

Tabela 3.1: Fontes de erro e mitigações no módulo ACC.

Fonte	Descrição	Mitigação
Baixa resolução	PDFs escaneados em < 150 DPI	Reprocessamento a 300 DPI
Tabelas complexas	Layout de certificado não padrão	Prompt com múltiplos formatos
Alucinação do modelo	LLM inventa carga não presente	Regex pós-geração + fallback manual
Idioma estrangeiro	Certificados em inglês/espanhol	Instrução multilíngue no prompt

3.5.2 Escalabilidade

O modelo é chamado uma vez por envio (PDF completo), tornando o custo fixo por aluno independente do número de páginas dentro do limite de contexto do Gemini 2.5 Flash Lite ($\approx 1,5 \times 10^6$ tokens). Para envios com muitos certificados, o sistema pode ser estendido com paginação em lotes de 10–15 páginas por chamada.

Capítulo 4

Infraestrutura e Integração

4.1 Banco de Dados Vetorial: LanceDB

LanceDB (LANCEDB, 2023) é um banco de dados vetorial embutido (*embedded*) baseado no formato Apache Lance, projetado para alto desempenho em hardware de commodity. No FasiTech, ele persiste:

- Tabela `recipes` — chunks dos documentos com vetores 768-d.
- Tabela `semantic_cache` — entradas de cache semântico com vetores, metadados e timestamps.

4.2 Modelos de Linguagem e Embedder

O FasiTech utiliza o framework **Agno** (ex-Phidata) para abstrair a camada de LLM. A seleção do modelo de geração é feita em tempo de inicialização do `ChatbotService`:

1. Se `GOOGLE_API_KEY` estiver configurada: usa **Gemini** via API Google.
2. Caso contrário: usa **Ollama** (OLLAMA, 2023) local como fallback (modelo `OLLAMA_LLM_MODEL`).

Tabela 4.1: Configuração dos modelos no FasiTech (`LLMConfig.py`).

Variável	Uso	Padrão
<code>GEMINI_MODEL</code>	Geração de respostas — Diretor Virtual	<code>gemini-2.5-flash</code>
<code>GEMINI_MODEL_VISION</code>	Extração de certificados — ACC	<code>gemini-2.5-flash-lite</code>
<code>OLLAMA_LLM_MODEL</code>	Fallback local (sem API key)	<code>qwen2.5:3b</code>
<code>nomic-embed-text</code>	Embeddings 768-d (sempre Ollama)	<code>OllamaEmbedder</code>

Nota: O embedder (`nomic-embed-text`) é sempre local via Ollama, independentemente da disponibilidade da API Google. O modelo de geração (Gemini vs. Ollama) é o único componente que muda conforme o ambiente.

4.3 Padrão de Projeto: Singleton com Inicialização Lazy

O `ChatbotService` implementa o padrão Singleton via `__new__`, garantindo que a inicialização custosa (conexão com Ollama, criação do índice LanceDB, carregamento dos documentos) ocorra apenas uma vez por processo. O diretório de dados é `~/.cache/fasitech/rag/` (persistência por usuário do SO, compatível com ambientes Docker).

Sessões de usuário são isoladas por `session_id`; cada sessão cria um `Agent Agno` sob demanda com histórico de conversa independente armazenado em SQLite (`.db` no mesmo diretório de cache). Quando `persist_history=False`, o histórico fica apenas em RAM.

Capítulo 5

Trabalhos Futuros

1. **Multi-Query Retrieval** — gerar 3 variações da pergunta via LLM antes da busca e fundir resultados com RRF (ES et al., 2023; LANGCHAIN, 2023).
2. **HyDE** (*Hypothetical Document Embeddings*) — embeddar uma resposta hipotética gerada pelo LLM em vez da query original para maior precisão semântica (GAO et al., 2023).
3. **Parent-Child Chunking** — indexar chunks pequenos para busca, retornar chunk pai para geração (LLAMAINDEX, 2023).
4. **Cross-Encoder Reranking** — aplicar um reranker (BAAI/bge-reranker-base) sobre os chunks recuperados antes de montar o prompt (KHATTAB; ZAHARIA, 2020).
5. **Interface de Feedback** — expor botões de avaliação (1–5 estrelas) no ChatWidget para alimentar o cache semântico com dados reais de usuários.
6. **Validação Cruzada ACC** — comparar resultado do Gemini com extração por OCR clássico (Tesseract) para medir concordância e detectar certificados problemáticos.

Referências

- ALIANNEJADI, Mohammad et al. ClariQ: Building a Dataset for Clarification of Questions in Open-Domain Information-Seeking Conversations. In: PROCEEDINGS of the 59th Annual Meeting of the Association for Computational Linguistics (ACL) Findings. [S.l.: s.n.], 2021. Disponível em: <<https://arxiv.org/abs/2009.11352>>.
- ANTHROPIC. **Introducing Contextual Retrieval**. [S.l.: s.n.], 2024. Blog post. Disponível em: <<https://www.anthropic.com/news/contextual-retrieval>>.
- BANG, Yue et al. **GPTCache: A Data Store for Efficient LLM Responses**. [S.l.: s.n.], 2023. <https://github.com/zilliztech/GPTCache>.
- BLECHER, Lukas et al. Nougat: Neural Optical Understanding for Academic Documents. In: ARXIV preprint. [S.l.: s.n.], 2023. Disponível em: <<https://arxiv.org/abs/2308.13418>>.
- BROWN, Tom B. et al. Language Models are Few-Shot Learners. **Advances in Neural Information Processing Systems (NeurIPS)**, v. 33, p. 1877–1901, 2020. Disponível em: <<https://arxiv.org/abs/2005.14165>>.
- CARPINETO, Claudio; ROMANO, Giovanni. A Survey of Automatic Query Expansion in Information Retrieval. **ACM Computing Surveys**, v. 44, n. 1, p. 1–50, 2012.
- CORMACK, Gordon V.; CLARKE, Charles L. A.; BUETTCHER, Stefan. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In: PROCEEDINGS of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval. [S.l.: s.n.], 2009. P. 758–759.
- CUI, Lei et al. Document AI: Benchmarks, Models and Applications. **arXiv preprint**, 2021. Disponível em: <<https://arxiv.org/abs/2111.08609>>.
- ES, Shahul et al. **RAGAS: Automated Evaluation of Retrieval Augmented Generation**. [S.l.: s.n.], 2023. <https://arxiv.org/abs/2309.15217>.
- GAO, Luyu et al. Precise Zero-Shot Dense Retrieval without Relevance Labels. In: PROCEEDINGS of the 61st Annual Meeting of the Association for Computational Linguistics (ACL). [S.l.: s.n.], 2023. Disponível em: <<https://arxiv.org/abs/2212.10496>>.

- GEMINI TEAM et al. Gemini: A Family of Highly Capable Multimodal Models. **arXiv preprint**, 2023. Disponível em: <<https://arxiv.org/abs/2312.11805>>.
- KARPUKHIN, Vladimir et al. Dense Passage Retrieval for Open-Domain Question Answering. In: PROCEEDINGS of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). [S.l.: s.n.], 2020. P. 6769–6781. Disponível em: <<https://arxiv.org/abs/2004.04906>>.
- KHATTAB, Omar; ZAHARIA, Matei. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In: PROCEEDINGS of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval. [S.l.: s.n.], 2020. P. 39–48. Disponível em: <<https://arxiv.org/abs/2004.12832>>.
- LANCEDB. **LanceDB: Developer-friendly, serverless vector database**. [S.l.: s.n.], 2023. <https://lancedb.github.io/lancedb/>.
- LANGCHAIN. **Multi-Query Retriever**. [S.l.: s.n.], 2023. https://python.langchain.com/docs/modules/data_connection/retrievers/MultiQueryRetriever.
- LEWIS, Patrick et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2020. v. 33, p. 9459–9474. Disponível em: <<https://arxiv.org/abs/2005.11401>>.
- LLAMAINDEX. **Small-to-Big Retrieval and Parent Document Retriever**. [S.l.: s.n.], 2023. https://docs.llamaindex.ai/en/stable/examples/retrievers/parent_child_retrieval.html.
- NUSSBAUM, Zach et al. **Nomic Embed: Training a Reproducible Long Context Text Embedder**. [S.l.: s.n.], 2024. <https://arxiv.org/abs/2402.01613>.
- OLLAMA. **Ollama: Get up and running with large language models locally**. [S.l.: s.n.], 2023. <https://ollama.ai>.
- ROBERTSON, Stephen E. et al. Okapi at TREC-3. In: PROCEEDINGS of the Third Text REtrieval Conference (TREC-3). [S.l.: s.n.], 1994.
- SHI, Freda et al. Large Language Models Can Be Easily Distracted by Irrelevant Context. In: PROCEEDINGS of the 40th International Conference on Machine Learning (ICML). [S.l.: s.n.], 2023. Disponível em: <<https://arxiv.org/abs/2302.00093>>.
- ZHU, Zhuoran et al. Semantic Cache for Large Language Model Inference with Similarity-Based Retrieval. **arXiv preprint**, 2023. Disponível em: <<https://arxiv.org/abs/2309.09558>>.